

LUIS ROMERO

AI Engineer | Backend Developer (Node.js/TypeScript/Python)

Mérida, Venezuela | +58 424 709 2980 | lueduar15@gmail.com

linkedin.com/in/luis-romero-dev-back15 | github.com/ROMEROLUIS15

PROFESSIONAL SUMMARY

AI Engineer and Backend Developer who ships AI products to production and keeps them running there. I currently build backend at a US software company, where I audited the product's backend, proposed its restructuring and am running it end to end. Alongside that I build and operate AI systems with real users every day: WhatsApp and voice agents that book appointments autonomously, a workshop assistant on Python/LangGraph serving both customers and back office, and an industrial assistant grounded in real service history. My work is not plugging in an LLM: it is designing the controls that stop it from inventing data or acting on the wrong record, and measuring its reliability with automated evaluations instead of assuming it.

EXPERIENCE

Backend Developer | Complexity (Texas, United States, Remote)

July 2026 - Present

Backend for a mobile application: versioned REST API with OpenAPI, real time over WebSockets, background job queues and full-text search, on a modular monolith of 26 domain modules (NestJS, PostgreSQL/TypeORM, Redis, Elasticsearch).

- I joined the team after passing their technical challenge: building a search and discovery engine for their product's real problem (NestJS + Elasticsearch), delivered deployed, documented with OpenAPI and security-scanned in CI — the detail is under PROJECTS.
- Audited the backend, presented my findings to the team and had my remediation plan approved; I am running it end to end. Exposure risks came first: I stripped sensitive data out of HTTP responses and logs, moved credentials that were embedded in the code into configuration validated at startup, and closed off access to the documentation in production. Every fix is pinned by new tests, and I repaired seven test suites that no longer compiled.
- Defined how the team works: spec-driven development with mandatory artifacts per change and traceability from ticket to PR — I write the specifications, implement them, and a second developer reviews before merging — plus branch and commit conventions, a PR template with a review budget that counts hand-written and generated code separately, and a CI workflow that verifies branch policy and independent review. The plan is broken down into ~150 scoped changes.

IBIME Connect (Institutional platform, Remote)

January 2025 - July 2026

Software Development Lead (May 2026 - July 2026) | Backend / Full-Stack Developer

(January 2025 - May 2026)

- Built the AI assistant a public library network uses to answer citizens' queries, inside the platform that replaced their legacy system. The institution approved it, adopted it as its official platform and, on the back of that delivery, appointed me Software Development Lead.
- Stopped the assistant from making up answers by keeping the model out of the decision flow: a rule-based intent classifier (no LLM) routes the conversation, and if the citizen's email already exists the answer comes from the database without calling the model — less cost, less latency and no opportunity to hallucinate on those queries. A deterministic chain (intent, data protection, match threshold, review and output policy) validates before responding.
- Implemented semantic search over the institution's documents: RAG with PostgreSQL + pgvector and 768-dim Gemini embeddings, inference on Groq gpt-oss-20b configurable per environment, Redis embedding cache, and a match threshold that prefers not answering over answering badly.
- Built a stateful LangGraph agent (extractor, validator and corrector, with bounded retries) that reads the institution's catalog PDFs and turns them into structured, validated data, discarding duplicates by content.
- Hardened the platform for handling citizens' personal data: blocked anonymous writes on sensitive tables (RLS), prevented duplicate registrations, and defined how long each piece of data is kept and how it is deleted. Hexagonal architecture with dependency injection, structured logs and error capture (Sentry); 420+ tests in CI.

Backend Developer | Kiura (Barranquilla, Colombia, Remote)

April 2024 - December 2024

- Built the backend services of a teletherapy platform: subscription payments that grant or suspend patient access (MercadoPago via webhook), real-time chat (Socket.IO) and video sessions with per-room signed access (Agora), over a 26-entity domain with dual authentication (JWT + Google OAuth) and role-based permissions.

AI PROJECTS

Founder & AI Engineer | Cronix (in production; operation and maintenance)

January 2026 - Present

Multi-tenant SaaS for appointment-based businesses, built and operated single-handedly. In production 24/7: 170+ real appointments handled by the agents and automated reminders, with no manual intervention. 1,600+ tests (Vitest, Playwright, pgTAP) and CI/CD with Sentry.

- Built the WhatsApp and voice agents that book, reschedule and cancel appointments on their own: the customer just asks, at any hour, and nobody at the business has to step in. Custom orchestration engine (no framework), with deterministic routes that resolve most cases without calling the model, and restricted tool access (Groq + Deepgram, 12 capabilities on the voice agent).
- Stopped the agent from inventing data or acting on the wrong customer — the real risk of giving an LLM write access to a database full of real clients — with 6 layers of control: keeping the model off the path when the route is deterministic, restricting which tools it may call and with what arguments, anchoring every value to what the user actually said, resolving names against a confidence threshold, forcing a two-step confirmation before any write, and a second model that reviews the operation and can veto it (LLM-as-judge).
- Built the system that checks whether the agent answers well: I saved real conversations as a benchmark and replay them on every change, so a deploy is blocked if the agent gets worse (DeepEval evals and golden conversations gating CI).
- Set up the agent's observability: every turn is logged with its outcome, tokens, latency (p50/p95) and errors, visible per business in a custom dashboard on PostgreSQL, with parallel fan-out to LangSmith and error alerting. All of it runs across 9 Edge Functions (Deno).
- Audited my own security until I found a hole in it: I discovered that 13 database functions let one business read another's data, reproduced the attack to confirm it, closed it with an ownership check, and locked it down with 147 automated security tests (pgTAP over 59 isolation policies).

AI Engineer / Backend | Motosmax Cordialidad (Client, Remote) (delivered in phases; in production with support)

June 2026 - Present

- Automated both customer service and back-office work for a motorcycle workshop with a Python microservice (FastAPI + LangGraph): one agent serves customers over WhatsApp and another assists the back office with session memory, custom tools, periodic reports and low-stock alerts. The model (Groq/DeepSeek, with fallback) is configurable per environment, and the WhatsApp channel handles Meta's 24h window and surfaces send failures.
- Wired it into the workshop's real system without handing it loose access: the AI microservice talks to the API (NestJS, hexagonal architecture, 37 data models) through credentials that expire in 5 minutes, with sensitive fields encrypted. 400+ tests (Jest, pytest, Playwright) with linting and strict typing in CI; every change starts from a versioned spec (Spec-Driven Development).

AI Engineer / Backend | CMMS Hidrobombas Merida (Client, Remote)
(delivered in phases; in production with support)

April 2026 - Present

- Built the AI assistant of an industrial maintenance company: it answers by consulting the real report history instead of improvising (LangGraph + RAG persisted on pgvector), with multi-graph orchestration — a tool-using agent plus a diagnostic graph — and the AI layer isolated behind ports so it can be tested without depending on the provider of the day (in-memory vector store in development, pgvector in production). 394 tests, 82% backend coverage.

Advanced Product Search API (The technical challenge to Complexity)
Live: advanced-search-api-chet.onrender.com/docs

July 2026

- Built the search that ranks results by what the person is actually looking for rather than by insertion order: BM25 relevance on Elasticsearch 8 with typo tolerance, adjusted by popularity and recency, with facet counts computed excluding their own filter (so the user can widen the search instead of hitting zero results), autocomplete and "did you mean" suggestions.
- Delivered it deployed and verified: 500+ tests (97% coverage, including integration against real Elasticsearch), a k6 load test (398,000 requests, 0 failures) and automated security in CI over code, dependencies, git-history secrets, the shipped image and the running API (CodeQL, gitleaks, Trivy + SBOM, OWASP ZAP).

TECHNICAL SKILLS

AI & LLM: LLM Integration, AI Agents, Multi-Agent Orchestration, LangGraph, LangChain, Tool Calling / Function Calling, RAG (Retrieval-Augmented Generation), Semantic Search, Embeddings, Vector Databases (pgvector), Prompt Engineering, Anti-Hallucination, AI Guardrails, LLM-as-a-Judge, LLM Evaluation (DeepEval, golden conversations), LLM Observability (LangSmith, traces, tokens, latency), MCP (Model Context Protocol), Voice AI (STT/TTS), Groq, DeepSeek, Gemini, Deepgram, n8n

Languages & frameworks: TypeScript, Node.js, NestJS, Python, FastAPI, Express, Deno, Next.js, React

Backend & data: REST APIs, OpenAPI/Swagger, Microservices, Serverless / Edge Functions, Event-Driven Architecture, Webhooks, Queues (BullMQ, QStash), Idempotency, Hexagonal Architecture, Clean Architecture, SOLID, Dependency Injection, PostgreSQL, SQL, pgvector, Redis, Prisma, TypeORM, Supabase, Neon, Elasticsearch, MySQL, Row-Level Security, Multi-tenant

DevOps, testing & security: Docker, GitHub Actions, CI/CD, Vercel, Render, Cloudflare (R2), Sentry, OpenTelemetry, Observability, Load Testing (k6), pytest, Vitest, Jest, Playwright, pgTAP, SAST/DAST/SCA (CodeQL, OWASP ZAP, Trivy, gitleaks), JWT, OAuth 2.0, RBAC, Rate Limiting, Security Auditing, Spec-Driven Development (SDD)

EDUCATION

- Engineering, Computer Systems area | UPTM Kleber Ramirez - Expected graduation: Dec 2026
- Full-Stack Bootcamp | Academlo - Mar 2024

LANGUAGES

- Spanish: Native
- English: Intermediate (B1+), advanced technical reading and writing